Student Course Management System

Project Submission Report

Full Name	Victor Oladimeji
Email Address	oladimejivictor611@gmail.com
Department	Software Engineering
Matriculation Number	2025/A/SENG/0149

1. What the System Does

I built this project to solve a problem I actually have every semester: keeping track of which courses I'm offsetting and how many units they add up to. SCM is a console-based Java program that does this through a simple numbered menu — I can add a course, view everything I've added, search for one by its code, check my total unit load, and save or reload that list from a text file so I don't lose it when I close the program.

It's split into three classes. `Main` just starts things off. `CourseManager` does the actual work — it runs the menu loop, keeps the list of courses in memory, and holds all the logic for adding, viewing, searching, saving and loading. `Course` is a small data class that just stores a course's code, title and unit value.

2. How Each Major Feature Works

2.1 Add Course

This asks for a course code, title and unit value, one at a time, and checks each one before accepting it: the code and title can't be blank, and the unit has to be a positive number. If something's wrong, I throw an `IllegalArgumentException` with a message that says exactly what failed, and a `catch` block around it prints that message instead of letting the program crash. If everything checks out, a new `Course` gets created and added to `courseList`.

2.2 View Courses

Prints every course in `courseList` as a formatted table, reusing `Course`'s own `toString()` so I'm not duplicating formatting logic. If the list is empty it just says so instead of printing an empty table, which looked weird when I first tested it.

2.3 Search Course

Takes a course code from the user and looks for a match using recursion instead of a plain loop (more on that in Section 4). If it finds the course it prints the details; if not, it tells the user plainly that nothing matched that code.

2.4 Compute Total Units

Loops through `courseList`, adds up every unit value, and prints the total. Nothing fancy here — it's just a quick way to see my total registered load at a glance.

2.5 Save Courses

Writes every course to `courses.txt` as a comma-separated line (code, title, unit) using a `BufferedWriter`, so the list survives after the program closes.

2.6 Load Courses

Reads `courses.txt` back in with a `BufferedReader`, one line at a time, splits each line on the comma, and rebuilds `courseList` from the three fields. It clears the in-memory list first so reloading doesn't duplicate courses I'd already added in the current session.

3. Challenges Encountered and How Errors Were Handled

The part that gave me the most trouble was getting saving and loading to work properly with `BufferedWriter` and `BufferedReader`. When I first tested it, reading the file back didn't always rebuild the list correctly — a line that didn't split into exactly three comma-separated pieces (say, a trailing blank line, or a row that got written before all three fields existed) would either get silently dropped or blow up when I tried to parse the unit as an integer. I fixed it by checking `data.length == 3` before building a `Course` from a line, so anything malformed just gets skipped instead of crashing the loader.

Beyond that specific bug, I wrapped every user-facing action in a `try/catch` so one bad input doesn't take down the whole program:

- **Bad input when adding a course** — empty code, empty title, or a unit that isn't positive — throws an `IllegalArgumentException` that `addCourse()` catches and reports back clearly.
- **Non-numeric menu choices** (typing a letter instead of a number) are caught in `showMenu()`, which just says the input was invalid and shows the menu again rather than crashing.
- **File problems** — no `courses.txt` yet on load, or a write failure on save — are caught as `IOException` so the program keeps running even before a save file exists.

4. Where Recursion Was Applied

I used recursion for the course search. `searchCourse()` hands off to a private helper, `findCourseByCode(String code, int index)`, which checks the course sitting at `index`:

- If `index` has run past the end of `courseList`, it returns `null` — base case, nothing found.
- If the course at `index` matches the code I'm looking for, it returns that course — base case, found it.
- Otherwise it calls itself again with `index + 1`, moving one step further into the list each time — the recursive case.

It steps through the list one course at a time until it either finds a match or runs out of courses to check. A loop would do the same job, but this was a good chance to apply recursion to something concrete instead of just a textbook example.

5. Program Screenshot

The screenshot below shows a real run of the program: adding two courses (CSC301 and MTH205), viewing the course list, searching for CSC301, computing total units, and exiting.

Main - java

```
==============================
 Victor's Student Course Management System
==============================
1. Add Course
2. View Courses
3. Search Course
4. Compute Total Units
5. Save Courses
6. Load Courses
7. Exit
Enter your choice: 1
Enter Course Code: CSC301
Enter Course Title: Data Structures and Algorithms
Enter Course Unit: 3
Course added successfully.

==============================
 Victor's Student Course Management System
==============================
1. Add Course
2. View Courses
3. Search Course
4. Compute Total Units
5. Save Courses
6. Load Courses
7. Exit
Enter your choice: 1
Enter Course Code: MTH205
Enter Course Title: Linear Algebra
Enter Course Unit: 2
Course added successfully.

==============================
 Victor's Student Course Management System
==============================
1. Add Course
2. View Courses
3. Search Course
4. Compute Total Units
5. Save Courses
6. Load Courses
7. Exit
Enter your choice: 2

-------------------------------------------
CODE       TITLE                          UNIT
-------------------------------------------
CSC301     Data Structures and Algorithms      3
MTH205     Linear Algebra                      2

==============================
 Victor's Student Course Management System
==============================
1. Add Course
2. View Courses
3. Search Course
4. Compute Total Units
5. Save Courses
6. Load Courses
7. Exit
Enter your choice: 3
Enter the Course Code to search for: CSC301

Course found:
CSC301     Data Structures and Algorithms      3

==============================
 Victor's Student Course Management System
==============================
```

```
==============================
1. Add Course
2. View Courses
3. Search Course
4. Compute Total Units
5. Save Courses
6. Load Courses
7. Exit
Enter your choice: 4
Total Units Across All Courses: 5

==============================
 Victor's Student Course Management System
==============================
1. Add Course
2. View Courses
3. Search Course
4. Compute Total Units
5. Save Courses
6. Load Courses
7. Exit
Enter your choice: 7
Goodbye!
```

Console output captured from running `Main.java`.

6. Source Code

Main.java

```
package SCM;

public class Main {

    public static void main(String[] args) {

        CourseManager manager = new CourseManager();

        manager.showMenu();

    }

}
```

Course.java

```
package SCM;

public class Course {

    private String courseCode;
    private String courseTitle;
    private int unit;

    public Course(String courseCode, String courseTitle, int unit) {
        this.courseCode = courseCode.trim().toUpperCase();
        this.courseTitle = courseTitle.trim();
        this.unit = unit;
    }

    public String getCourseCode() {
        return courseCode;
    }

    public String getCourseTitle() {
        return courseTitle;
```

```
    }

    public int getUnit() {
        return unit;
    }

    public void setCourseCode(String courseCode) {
        this.courseCode = courseCode.trim().toUpperCase();
    }

    public void setCourseTitle(String courseTitle) {
        this.courseTitle = courseTitle.trim();
    }

    public void setUnit(int unit) {
        this.unit = unit;
    }

    @Override
    public String toString() {
        return String.format("%-10s %-35s %d",
                courseCode,
                courseTitle,
                unit);
    }
}
```

CourseManager.java

```
package SCM;

import java.io.*;
import java.util.ArrayList;
import java.util.Scanner;

public class CourseManager {

    private ArrayList<Course> courseList = new ArrayList<>();
    private Scanner scanner = new Scanner(System.in);

    public void showMenu() {

        System.out.println("\n==============================");
        System.out.println(" Victor's Student Course Management System");
        System.out.println("==============================");
        System.out.println("1. Add Course");
        System.out.println("2. View Courses");
        System.out.println("3. Search Course");
        System.out.println("4. Compute Total Units");
        System.out.println("5. Save Courses");
```

```
        System.out.println("6. Load Courses");
        System.out.println("7. Exit");
        System.out.print("Enter your choice: ");

        try {

            int choice = Integer.parseInt(scanner.nextLine());

            switch (choice) {

                case 1:
                    addCourse();
                    break;

                case 2:
                    viewCourses();
                    break;

                case 3:
                    searchCourse();
                    break;

                case 4:
                    displayTotalUnits();
                    break;

                case 5:
                    saveCourses();
                    break;

                case 6:
                    loadCourses();
                    break;

                case 7:
                    System.out.println("Goodbye!");
                    return;

                default:
                    System.out.println("Invalid choice. Please enter a number
between 1 and 7.");
            }

        } catch (Exception e) {
            System.out.println("Invalid input. Please enter a number.");
        }

        showMenu();
    }

    public void addCourse() {

        try {

            System.out.print("Enter Course Code: ");
            String code = scanner.nextLine().trim().toUpperCase();
```

```
            if(code.isEmpty()){
                throw new IllegalArgumentException("Course code cannot be
empty.");
            }

            System.out.print("Enter Course Title: ");
            String title = scanner.nextLine().trim();

            if(title.isEmpty()){
                throw new IllegalArgumentException("Course title cannot be
empty.");
            }

            System.out.print("Enter Course Unit: ");
            int unit = Integer.parseInt(scanner.nextLine());

            if(unit<=0){
                throw new IllegalArgumentException("Unit must be a positive
number.");
            }

            courseList.add(new Course(code,title,unit));

            System.out.println("Course added successfully.");

        }catch(Exception e){

            System.out.println("Could not add course: "+e.getMessage());

        }

    }

    public void viewCourses(){

        if(courseList.isEmpty()){

            System.out.println("No courses have been added yet.");
            return;

        }

        System.out.println("\n-----------------------------------------------");
        System.out.printf("%-10s %-35s %s%n",
                "CODE","TITLE","UNIT");
        System.out.println("-----------------------------------------------");

        for(Course course : courseList){

            System.out.println(course);

        }

    }
```

```
    public void searchCourse(){

        System.out.print("Enter the Course Code to search for: ");

        String code = scanner.nextLine().trim().toUpperCase();

        Course foundCourse = findCourseByCode(code,0);

        if(foundCourse==null){

            System.out.println("No course found with code \""+code+"\".");

        }else{

            System.out.println("\nCourse found:");
            System.out.println(foundCourse);

        }

    }

    private Course findCourseByCode(String code, int index){

        if(index>=courseList.size())
            return null;

        if(courseList.get(index).getCourseCode().equalsIgnoreCase(code))
            return courseList.get(index);

        return findCourseByCode(code,index+1);

    }

    public void displayTotalUnits(){

        int totalUnits=0;

        for(Course course : courseList){

            totalUnits+=course.getUnit();

        }

        System.out.println("Total Units Across All Courses: "+totalUnits);

    }

    public void saveCourses(){

        try{

            BufferedWriter writer=new BufferedWriter(new
FileWriter("courses.txt"));

            for(Course course : courseList){
```

```
                writer.write(course.getCourseCode()+","
                        +course.getCourseTitle()+","
                        +course.getUnit());

                writer.newLine();

            }

            writer.close();

            System.out.println("Courses saved successfully to courses.txt.");

        }catch(IOException e){

            System.out.println("Error saving courses to file.");

        }

    }

    public void loadCourses(){

        try{

            BufferedReader reader=new BufferedReader(new
FileReader("courses.txt"));

            courseList.clear();

            String line;

            while((line=reader.readLine())!=null){

                String[] fields=line.split(",");

                if(fields.length==3){

                    courseList.add(

                            new Course(

                                    fields[0],

                                    fields[1],

                                    Integer.parseInt(fields[2])

                            )

                    );

                }

            }

            reader.close();
```

```
            System.out.println("Courses loaded successfully from courses.txt.");
        }catch(IOException e){
            System.out.println("No saved course file found.");
        }
    }
}
```